

兆鸣嵌入式

C语言·一个LED讲透面向对象

EP10 HAL库几千个函数，就一个套路

三家世界500强 · 11年一线嵌入式实战

公众号搜索「兆鸣嵌入式」回复「交流」进群

核心概念

这一期是"验证课"——不教新概念。EP06-EP09学的所有东西，在STM32 HAL库里都有一一对应。你学的不是"教学用的小技巧"，而是工业界几十年沉淀的标准做法。

几千个函数——就这一招

设计思路

定位：验证课，不教新概念

用STM32

HAL库的GPIO模块做活教材，把你在EP06-EP09学的struct封装、me指针、前缀命名、init/deinit、头文件/static、数据归位——全部在真实工业代码里找到对应。让你确认：我学的就是行业标准。

核心：六个映射

1. GPIO_TypeDef = struct封装数据 (EP06)
2. GPIOA/GPIOB/GPIOC = 多实例 (EP06)
3. GPIO_TypeDef *GPIOx = me指针 (EP06)
4. HAL_GPIO_前缀 = 类名 (EP08)
5. HAL_GPIO_Init/DelInit = 构造/析构 (EP08)
6. .h声明 + .c的static = 菜单 + 后厨 (EP07)

视频讲义

HAL 库的 GPIO 定义

打开源码，看看 GPIO_TypeDef 长什么样

HAL 库源码 GPIO_TypeDef

```
typedef struct {
    __IO uint32_t MODER;      // 模式寄存器
    __IO uint32_t ODR;        // 输出数据寄存器
    __IO uint32_t BSRR;       // 置位/复位寄存器
    ...                       // 其他寄存器
} GPIO_TypeDef;
```

本质

就是一个 **struct**

作用

把一个 GPIO 端口的
所有寄存器打包在一起

你学过的

和你的 **Led struct**
一模一样的思路

属于一个对象的数据，装进一个 struct

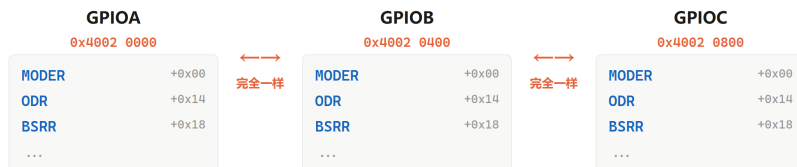
嵌入式兆鸣

GPIO_TypeDef: 和Led_t一样的struct封装

同一份布局，复制粘贴三份

芯片内部的 GPIO 模块 —— 寄存器完全一样，只是地址不同

芯片内部 三个 GPIO 端口 —— 寄存器布局完全相同



你的代码 一个 struct 定义，三个指针 —— 不复制粘贴代码

```
#define GPIOA ((GPIO_TypeDef *) 0x40020000) // 指向 A 端口
#define GPIOB ((GPIO_TypeDef *) 0x40020400) // 指向 B 端口
#define GPIOC ((GPIO_TypeDef *) 0x40020800) // 指向 C 端口
```

唯一的区别是地址 —— 把不变的统一，把变化的抽出来

嵌入式兆鸣

GPIOA/B/C的内存布局：同struct不同地址 = 多实例

HAL 库的函数

前缀、Init、me 指针 —— 全是你学过的

HAL 库函数

GPIO 操作函数

```
void HAL_GPIO_Init (GPIO_TypeDef *GPIOx, GPIO_InitTypeDef *cfg);
void HAL_GPIO_DeInit (GPIO_TypeDef *GPIOx, uint32_t Pin);
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t Pin, uint8_t State);
void HAL_GPIO_ReadPin (GPIO_TypeDef *GPIOx);
```

橙色部分

HAL_GPIO_

前缀 = 类名

绿色部分

Init / DeInit

构造函数 / 析构函数

紫色部分

GPIO_TypeDef *GPIOx

me 指针 = 操作哪个 GPIO

.h 头文件

只写函数声明 = 菜单

.c 源文件

static 辅助函数 = 后厨

嵌入式兆鸣

函数映射：led_init vs HAL_GPIO_Init

六个概念 —— 一一对应

你学的每一个封装概念，在 HAL 库里都有

你学的	HAL 库里的	来源
struct 封装数据	GPIO_TypeDef	struct + 打包
多实例	GPIOA / GPIOB / GPIOC	一份代码，多份数据
me 指针	GPIO_TypeDef *GPIOx	告诉函数“操作谁”
前缀 = 类名	HAL_GPIO_	前缀 = 类名
Init / DeInit	HAL_GPIO_Init / DeInit	构造函数 / 析构函数
.h = 菜单 static = 后厨	.h 声明 + .c 内 static	公开接口 + 隐藏实现

不只 GPIO UART_TypeDef SPI_TypeDef TIM_TypeDef —— 每个外设都是同一个套路

几千个函数 —— 就这一招

嵌入式兆鸣

完整映射总表：你学的 = HAL库的

一张图看完 HAL 库的设计

每一处高亮，都是你学过的概念

□ 前缀 = 类名 □ Init / DeInit = 构造 / 析构 □ GPIO_TypeDef *GPIOx = me 指针

HAL 库完整函数 GPIO 模块 —— 所有函数一览

```
void HAL_GPIO_Init (GPIO_TypeDef *GPIOx, GPIO_InitTypeDef *cfg);
void HAL_GPIO_DeInit (GPIO_TypeDef *GPIOx, uint32_t Pin);
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t Pin, uint8_t State);
void HAL_GPIO_ReadPin (GPIO_TypeDef *GPIOx);
void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t Pin);
```

多实例 同一个 struct，不同的地址 —— 一份代码驱动所有 GPIO

```
#define GPIOA ((GPIO_TypeDef *) 0x40020000) // A 端口
#define GPIOB ((GPIO_TypeDef *) 0x40020400) // B 端口
#define GPIOC ((GPIO_TypeDef *) 0x40020800) // C 端口
```

几千个函数 —— 就这一招

嵌入式兆鸣

EP06-EP09知识总览：全部映射到HAL库

代码结构

文件	说明
gpio_typedef.h	GPIO寄存器结构体 + 端口实例（对应EP06）
hal_gpio.h	HAL GPIO公共接口——菜单（对应EP07+EP08）
hal_gpio.c	HAL GPIO实现——后厨 + static辅助函数
main.c	演示六个映射关系的完整demo
Makefile	编译脚本

关键代码解读

1. GPIO_TypeDef——和Led_t一样的struct封装（EP06）

gpio_typedef.h

```
typedef struct {
    uint32_t MODER; /* 模式寄存器 */
    uint32_t OTYPER; /* 输出类型寄存器 */
    uint32_t OSPEEDR; /* 输出速度寄存器 */
    uint32_t PUPDR; /* 上下拉寄存器 */
    uint32_t IDR; /* 输入数据寄存器 */
    uint32_t ODR; /* 输出数据寄存器 */
    uint32_t BSRR; /* 置位/复位寄存器 */
    uint32_t LCKR; /* 锁定寄存器 */
} GPIO_TypeDef;
```

和Led_t打包pin/brightness/is_on是一个道理——struct封装相关数据。

2. GPIOA/GPIOB/GPIOC——多实例 + me指针 (EP06)

gpio_typedef.h

```
/* 三个GPIO端口实例——同struct不同数据 */
extern GPIO_TypeDef g_gpioa_regs;
extern GPIO_TypeDef g_gpiob_regs;
extern GPIO_TypeDef g_gpioc_regs;

#define GPIOA (&g_gpioa_regs)
#define GPIOB (&g_gpiob_regs)
#define GPIOC (&g_gpioc_regs)

/* 真实STM32:
#define GPIOA ((GPIO_TypeDef *)0x40020000UL) */
```

和red_led/green_led是一个道理：同一个struct定义，不同的实例。真实芯片上是不同的硬件地址，我们用全局变量模拟。

3. HAL_GPIO_Init——me指针 + 构造函数 (EP06+EP08)

hal_gpio.c

```
void HAL_GPIO_Init(GPIO_TypeDef *GPIOx,
                    GPIO_InitTypeDef *init)
{
    if (GPIOx == NULL || init == NULL) return;

    for (int pin = 0; pin < 16; pin++) {
        if (!(init->Pin & (1U << pin))) continue;
        set_2bit_field(&GPIOx->MODER, pin, init->Mode);
        set_1bit_field(&GPIOx->OTYPER, pin, init->OType);
        set_2bit_field(&GPIOx->OSPEEDR, pin, init->Speed);
        set_2bit_field(&GPIOx->PUPDR, pin, init->Pull);
    }
}
```

GPIOx就是me指针——“我操作的是哪个端口”。和led_init(Led_t *me, uint8_t pin)是一个套路。

映射总表：你学的 = HAL库的

你学的	HAL库里的	出自
Led_t (struct)	GPIO_TypeDef (struct)	EP06 封装
red_led / green_led	GPIOA / GPIOB / GPIOC	EP06 多实例
Led_t *me	GPIO_TypeDef *GPIOx	EP06 me指针
led_前缀	HAL_GPIO_前缀	EP08 类名
led_init / deinit	HAL_GPIO_Init / DeInit	EP08 构造/析构
.h声明 + .c的static	菜单 + 后厨	EP07 信息隐藏

与真实HAL库的关系

这个demo是教学用的迷你版本，在PC上编译运行。接口设计（函数签名、参数结构体、命名规范）与真实STM32

HAL库完全一致。区别只在于：真实HAL库操作的是硬件寄存器地址，我们用全局变量模拟。设计模式完全相同——因为套路完全相同。

编译运行

```
cd code/EP10_HAL映射  
make  
./demo
```

获取更多

公众号搜索「兆鸣嵌入式」

回复「交流」加群，与一线工程师切磋

抖音 / B站 / 视频号

搜索「兆鸣嵌入式」